MORETON CAPITAL PARTNERS

Trading Desk: Trader Guide

How to use the Trading Desk API as a trader: onboarding, market data, orders, trades and balance. The `trading-desk-trader` Postman collection is a runnable version of this guide, and every step below exists as a numbered request in it.

Contents

1 Setup

1. Import `trading-desk-trader.postman_collection.json` into Postman.
2. Under the collection's **Variables** tab, set `baseUrl` (the API URL you were given), `userEmail` and `userPassword`.
3. Before the market/order requests, also set `tokenId` (an outcome token) and `conditionId` (its market); copy them from a **List markets** response.
4. Run the requests top to bottom: test scripts auto-capture tokens and ids into collection variables. Polling requests say so in their name: re-send until they pass.

2 How authentication works

- `register`, `login` and `/health` are public (login/register rate-limited 5/min per IP). Everything else needs `Authorization: Bearer <JWT>` from login.
- Trading endpoints additionally require you to be **onboarded**: credentials provisioned and a deposit wallet deployed by an admin. If anything is missing you get a single `403` naming what's absent.
- Your credentials are cached server-side for the token's lifetime (24 h) and re-warmed on every login; if trading calls return `403` for unavailable credentials, just log in again.
- `POST /v1/auth/logout` invalidates the current token.

3 Getting started (first time)

Step	Do	Expect
1	`POST /v1/auth/register` with `email` + `password` (skip if an admin registered you)	`201`, `credential_status: pending`
2	Wait for an admin to onboard you; poll `GET /v1/users/me`	`credential_status: ready`
3	`POST /v1/auth/login`	`200` + JWT (`409` = not provisioned yet)
4	`GET /v1/balance`	your funded pUSD balance

4 Everyday tasks

- **Check balance** → `GET /v1/balance`. After an external deposit, force a refresh first: `POST /v1/balance/sync`.

- **Find a market** → `GET /v1/markets?keyword=...`, then `GET /v1/markets/clob-info/{conditionId}` for tick size, fees, neg-risk flag and outcome tokens in one call.
- **Place a limit order** → check `GET /v1/markets/orderbook/{tokenId}`, then `POST /v1/orders`.
- **Place a market order** → `GET /v1/markets/price-estimate/{tokenId}` FIRST, then `POST /v1/orders/market`, which fills immediately.
- **Cancel** → one order `DELETE /v1/orders/{id}` · a list `POST /v1/orders/cancel-batch` · one market `POST /v1/orders/cancel-market` · everything (kill-switch) `POST /v1/orders/cancel-all`.
- **See your fills** → `GET /v1/trades`.

⚠ REAL MONEY

- Orders trade real pUSD on Polymarket. The CLOB enforces ~$1 minimum notional (`price × size ≥ 1`).
- Market orders (FOK) take liquidity and fill the moment they match; always price-estimate first.
- **On a BUY, `size` is a floor, not a target.** You commit at most `price × size` pUSD and receive at least `size` shares, so the spend is the hard cap and the share count can come out higher. Measured: `size: 3` at `price: 0.90` into a ~0.57 book filled 4.74 shares for $2.70. For an exact share count, price at the best ask from `GET /v1/markets/orderbook/{tokenId}` rather than above it. A SELL is exact: `size` shares leave, and the proceeds are at least `price × size`.
- **Fees are charged on top.** `amount` on a market order is the notional, not the debit: a $1.00 BUY of a longshot was debited $1.0595. You pay `fee_rate × (1 − price)` of what you spend; get `fee_rate` from `GET /v1/markets/fee-rate/{tokenId}`. Only takers pay, so a limit order that rests on the book costs nothing.
- Network retries are safe when you send a `client_order_id`: repeating it replays the stored result instead of placing a second order.

5 Endpoint reference

5.1 Onboarding & session

Register

`POST` `/v1/auth/register` `PUBLIC`

Inputs: body `email`, `password`

Auth: none (public, rate-limited 5/min per IP shared with login).

Creates the user **identity only** (no wallet, no Polymarket credentials). Returns `201` with a JWT and `credential_status: pending`; an admin must then provision credentials before login works. Skip this request if an admin registered you.

Password rules: 8–72 chars, ASCII letters, digits and special characters only (no accents, emoji or spaces); at least one uppercase, one lowercase, one digit and one special character.

Errors: `409` email already registered · `422` email malformed / password rules not met · `429` rate-limited.

Get me

`GET` `/v1/users/me` `BEARER`

Your own profile, the PII-free public shape: `id`, `email`, `created_at`, `role`, `credential_status`, `account_status`. While waiting for an admin to provision you, re-send this request until `credential_status` reads `ready`, then log in.

Errors: `401` missing/expired/blacklisted token.

Login

`POST` `/v1/auth/login` `PUBLIC`

Inputs: body `email`, `password`

Auth: none (public, rate-limited).

Returns `200` with a fresh JWT once credentials are `ready`, and re-warms the server-side credential cache that the trading endpoints need (cached for the token's 24 h lifetime; if trading calls ever return 403 for unavailable credentials, log in again). The server records your IP: `last_login_ip` on every login, `registration_ip` once at first login.

Errors: `401` bad credentials · `403` account disabled · `409` credentials not ready · `429` rate-limited.

Health check

`GET` `/health` `PUBLIC`

Auth: none (the only unauthenticated path).

Aggregated db / redis / Polymarket checks: `{status: ok|degraded, service, version, checks: {...}}`. Always returns `200`; a failing dependency shows up as `status: degraded` in the body, never as a 5xx.

5.2 Markets: read-only (no funds)

List markets

`GET` `/v1/markets` `ONBOARDED`

Inputs: query `limit`, `active`, `closed`, `keyword`

Active markets from the Polymarket Gamma API.

Query: `limit`, `active`, `closed`, `keyword` (all optional).

Order book

`GET` `/v1/markets/orderbook/{tokenId}` `ONBOARDED`

The CLOB order book (bids/asks) for one outcome token.

Price estimate

`GET` `/v1/markets/price-estimate/{tokenId}` `ONBOARDED`

Inputs: query `side`, `amount`, `order_type`

Estimated execution price for a market order, computed from the live book; check this before any FOK order.

Query: `side` (buy|sell), `amount` (pUSD to spend on buy / shares on sell), `order_type` (FOK|FAK).

Execution price only, fees excluded. The taker fee is charged on top of the fill; combine this with "Fee rate" for the total debit.

Errors: `422` no matchable liquidity for that size.

Tick size

`GET` `/v1/markets/tick-size/{tokenId}` ONBOARDED

Minimum price increment for the token; order prices must be a multiple of this.

Fee rate

`GET` `/v1/markets/fee-rate/{tokenId}` ONBOARDED

The taker-fee parameters for the token's market: `fee_rate`, `fee_exponent`, `taker_only` and the resolved `condition_id`. `fee_rate: 0` means the market is free; Polymarket omits the fee block entirely on those.

What you pay. Takers are charged `shares × fee_rate × price × (1 − price)` in pUSD, **on top of** the trade. As a share of what you spend that reduces to `fee_rate × (1 − price)`: at `fee_rate` 0.07, price 0.15 costs ~6%, price 0.60 costs ~2.8%. Dearest on longshots, near zero on heavy favourites. Makers pay nothing, so a limit order that rests on the book is free.

Rates observed: 0.07 crypto, 0.05 sports and most others, no fee on geopolitical. Ignore the `base_fee` on Polymarket's own `/fee-rate`: it is a flat 1000 on every market and prices nothing.

Errors: `404` unknown token id.

Last trade price

`GET` `/v1/markets/last-trade-price/{tokenId}` ONBOARDED

Most recent trade price for the token.

CLOB market info

`GET` `/v1/markets/clob-info/{conditionId}` ONBOARDED

Bundled CLOB market params by condition id: tick size, neg-risk flag, fees, outcome tokens, rewards: one call instead of four.

Errors: `404` unknown condition id.

Market detail

`GET` `/v1/markets/detail/{conditionId}` ONBOARDED

Single market detail by condition id (`get_market`).

Errors: `404` unknown condition id.

5.3 Orders: limit (⚠ real funds)

Place limit order

POST `/v1/orders` ONBOARDED

Inputs: body `token_id`, `side`, `price`, `size`, `order_type`, `client_order_id`

Places a GTC limit order via a POLY_1271 client (your EOA signs, your deposit wallet funds).

Body: `token_id`, `side` (buy|sell), `price` (0<p<1, ≤4 decimals, multiple of tick size), `size` (≤6 dp), `order_type` (GTC|GTD; GTD also needs `expiration` > now+60 s), optional `client_order_id` (8–128 chars): retries with the same value replay the stored result instead of re-placing.

What `size` commits you to. The order is signed as a ratio, not a share count. On a **BUY** you put up at most `size × price` pUSD and must get back at least `size` shares: the spend is the hard cap, `size` is only a floor. A **SELL** mirrors it: exactly `size` shares leave and you receive at least `size × price`. Measured against Polymarket (they document this nowhere): a BUY priced above the best ask spends the whole `size × price` and returns more than `size` shares: `size: 3` at `price: 0.90` into a ~0.57 book filled **4.74 shares for $2.70**. For an exact share count, price at the best ask from `GET /v1/markets/orderbook/{tokenId}`, never above it.

Fees. Takers only. A limit order that rests on the book is charged nothing; one that crosses and fills immediately pays the taker fee on top of the trade; see "Fee rate".

Errors: `422` local validation or CLOB rejection (Polymarket's message forwarded) · `409` same client_order_id in flight · `502` CLOB unreachable.

List open orders

GET `/v1/orders` ONBOARDED

Inputs: query `market`, `asset_id`

Your open orders, live from the CLOB (never the local DB; the CLOB is the source of truth). Scoped to your own API credentials.

Query: optional `market`, `asset_id` filters.

Get the order

GET `/v1/orders/{clobOrderId}` ONBOARDED

One order by CLOB id, live passthrough.

Errors: `404` unknown id.

Cancel the order

DELETE `/v1/orders/{clobOrderId}` ONBOARDED

Cancels the order. Idempotent-safe by design: an already-gone order is `200 {canceled: false, reason}`, never an error, so retry loops are harmless.

Errors: `502` only when the CLOB itself is unreachable.

5.4 Market order (⚠ real funds, fills immediately)

Place FOK market order

POST `/v1/orders/market` ONBOARDED

Inputs: body `token_id`, `side`, `amount`, `order_type`

Places a fill-or-kill market order.

Body: `token_id`, `side`, `amount` (pUSD to spend on BUY, shares to sell on SELL), `order_type` (FOK|FAK), optional `client_order_id` (same replay semantics as limit orders). The audit row stores `price: null` (the book prices it).

`amount` is the notional, not the debit. The taker fee is charged on top of `amount`, so more leaves your wallet than you asked to spend: a `$1.00` BUY was debited **$1.0595** in testing. Budget roughly `amount × (1 + fee_rate × (1 − price))` and leave headroom: a BUY for your entire balance is rejected because the fee cannot be covered. See "Fee rate".

Errors: `422` validation / CLOB rejection (e.g. not enough balance or liquidity) · `502` CLOB unreachable.

5.5 Bulk cancels

Cancel a batch

POST `/v1/orders/cancel-batch` ONBOARDED

Inputs: body `order_ids`

Cancels a list of orders by CLOB id (min 1). Unknown/already-gone ids come back under `not_canceled` with reasons, not an error.

Cancel all (kill-switch)

POST `/v1/orders/cancel-all` ONBOARDED

Cancels ALL of your open orders across every market. The kill-switch.

Cancel one market

POST `/v1/orders/cancel-market` ONBOARDED

Inputs: body `market`

Cancels all of your orders in one market.

Body: `market` (condition id, required), optional `asset_id` to narrow to one token.

5.6 Trades & balance (read-only)

Trade history (one page per call)

GET `/v1/trades` ONBOARDED

Inputs: query `market`, `asset_id`, `before`, `after`, `next_cursor`

Your fills, paginated explicitly: one page per call, pass back `next_cursor` from the previous response for the next page.

Query: optional `market` (condition id), `asset_id` (token id), `before` / `after` (unix seconds), `next_cursor`.

Balance & allowance (pUSD)

GET `/v1/balance` ONBOARDED

Inputs: query `asset_type`, `token_id`

Your deposit wallet's balance and allowance as the order book sees it.

Query: `asset_type` = `collateral` (default, pUSD) or `conditional` (which also requires `token_id`, else `422`).

Sync balance from on-chain

POST `/v1/balance/sync` ONBOARDED

Inputs: query `asset_type`, `token_id`

Forces the CLOB to refresh its cached view of your balance from the chain; run after external transfers into the deposit wallet.

Query: same `asset_type` / `token_id` semantics as the balance request.

5.7 Session end

Logout

POST `/v1/auth/logout` BEARER

Blacklists this token's `jti` for its remaining lifetime: the single-session logout.